Free Value Or Else

Steve Downey <sdowney@gmail.com>

Document #:	D4270R0
Date:	2026-06-12
Project:	Programming Language C++
Audience:	LEWG

Abstract

A set of free functions implementing the pattern of optional and expected `value_or` as free functions that produce the correct common reference type that was unfixable for `std::optional::value_or`.

Contents

Changes Since Last Version

— **R0** Initial revision.

1 Comparison table

1.1 Raw pointer with a manual null check

The fundamental nullable type is the raw pointer, and the fundamental idiom for providing a default is the conditional expression guarded by a null check. The free function names the idiom, removes the repetition of the checked entity, and computes the type of the result correctly.

```
Cat* cat = find_cat("Fido");
return cat ? *cat : Cat{"Default"};
```

```
Cat* cat = find_cat("Fido");
return std::value_or(cat, Cat{"Default"});
```

1.2 The `value_or` truncation pitfall

Because `std::optional<T>::value_or` converts its argument to `T`, a perfectly reasonable-looking sentinel silently truncates. The free function computes the common type of the contained value and the alternative, and converts both *outward* to that type instead of forcing the alternative *inward* through `T`.

```
std::optional<std::uint16_t> o = get();
auto v = o.value_or(-1);
// v is uint16_t{65535}
```

```
std::optional<std::uint16_t> o = get();
auto v = std::value_or(o, -1);
// v is int; -1 is preserved
```

1.3 Reference alternative without copies

When both the contained value and the alternative are references to long-lived objects, the result should be a reference. Member `value_or` cannot do this; `reference_or` can, and statically rejects the cases where the result would dangle.

```
std::optional<Logger&> opt = get_logger();
Logger& l =
    opt ? *opt : default_logger();
```

```
std::optional<Logger&> opt = get_logger();
Logger& l = std::reference_or(
    opt, default_logger());
```

2 Motivation

`std::optional<T>::value_or` answers the most common question asked of an optional — "the value, or this instead" — and answers it with the wrong type. It returns `T`, and it requires the alternative to be convertible to `T`. Every use therefore funnels both the contained value and the alternative through `T`, whether or not `T` is the right type for the result. The consequences are familiar: sentinels truncate (`optional<uint16_t>.value_or(-1)` is `65535`), wider alternatives narrow, and reference results are simply impossible, so `value_or` forces a copy even when both candidate results are references to long-lived objects.

This is not fixable in place. Changing the return type of a member function that has been shipping since C++17 is both an API and an ABI break, and the &&-qualified overload set compounds the problem. During the work on `std::optional<T&>` [P2988R12] the question was reopened in earnest, because for an optional reference the cost of the wrong answer is no longer a pessimization but a wrong program: returning `T` from `optional<T&>::value_or` copies, returning `T&` dangles when the alternative is a temporary. After extensive discussion in LEWG, P2988 concluded that "there is no particularly wonderful solution for `value_or` that does not involve a time machine," adopted the least objectionable member behavior — return `remove_cv_t<T>` by value — and explicitly forecast this paper: free functions over all types modeling an optional-like concept, where the return type can finally be computed correctly.

Existing practice diverges precisely because there is no one member-function answer that satisfies everyone:

Implementation	Behavior
Standard	`optional<T>::value_or` returns a `T`
Boost	`optional<T>::value_or` returns a `T`
	`optional<T&>::value_or` returns a `T&`
TartanLlama	`optional<T>::value_or` returns a `T`
	`optional<T&>::value_or` returns a `T`
Flux	returns the type of the ternary, similar to `common_reference`
Think-Cell	returns `common_reference`, with some caveats

The machinery to compute the right answer postdates `optional`. `std::common_type` existed but was not used; `std::common_reference` arrived with ranges in C++20; and `std::reference_constructs_from_temporary_v` [P2255R2] arrived in C++23, making it possible for the first time to *statically reject* exactly the dangling cases that made reference-returning `value_or` untenable. The conditional expression `b ? *m : u` has computed the common type all along; the library can now name that computation, add the lifetime check the core language does not perform, and do so once, for every nullable type.

That last point is the generalization this paper takes from [P1255R13]: being "contextually convertible to `bool` and dereferenceable" is a protocol, not a property of `std::optional`. Raw pointers, `std::shared_ptr`, `std::unique_ptr`, `std::expected<T,E>`, and `std::optional<T&>` all speak it. Each of them today requires either a hand-written conditional or a type-specific idiom to express "the value, or this instead." A single constrained free function family covers them all, with one set of semantics to learn and one place to get the corner cases right.

3 Design

The proposal is three free function templates over a `nullable` concept:

```
template <class T>
concept nullable = requires(const T t) {
    bool(t);
    *(t);
};

template <nullable T, class U,
          class R = std::common_reference_t<std::iter_reference_t<T>, U&&>>
constexpr auto reference_or(T&& m, U&& u) -> R;

template <nullable T, class U,
          class R = std::common_type_t<std::iter_reference_t<T>, U&&>>
constexpr auto value_or(T&& m, U&& u) -> R;

template <nullable T, class I,
          class R = std::common_type_t<std::iter_reference_t<T>,
                                       std::invoke_result_t<I>>>
constexpr auto or_invoke(T&& m, I&& invocable) -> R;
```

All three have the same engaged-path behavior — produce `*m`, converted to `R` — and differ in how the alternative is supplied (a value, a value bound as a reference, a nullary invocable) and in how `R` is computed (`common_type` or `common_reference`).

3.1 The `nullable` concept

The concept is the "maybe protocol" of [P1255R13]: an object is nullable if it is contextually convertible to `bool` and dereferenceable. The requirements are stated on `const T`: observation must not require mutable access. A type whose `operator bool` or `operator*` is non-const does not model `nullable`; asking "the value or a default" is a query, and a type that cannot be queried through const access is not answering that question.

Types verified to model the concept: `std::optional<T>`, `std::expected<T,E>`, raw object pointers `T*`, `std::shared_ptr<T>`, `std::unique_ptr<T>`, and `std::optional<T&>` as specified by [P2988R12]. Types verified not to model it: arithmetic types and `bool` (contextually convertible to `bool` but not dereferenceable), `std::string` and the containers (neither), `std::nullptr_t`, `void`, and types providing only one of the two operations. Iterators in general do not model `nullable` because they are not contextually convertible to `bool` — correctly so, since an iterator's validity is not a property the iterator itself can report.

One refinement remains open. P1255's `nullable_object` additionally requires that the result of dereferencing be an equality-preserving *object*, which excludes function pointers; the minimal concept above admits them (a function pointer is contextually convertible to `bool` and dereferenceable), and they are then rejected only downstream, when `common_type` fails. LEWG guidance is sought on whether the concept should require `is_object_v` of the referent, rejecting function pointers at the constraint rather than in the return-type computation.

3.2 Return type computation

`std::iter_reference_t<T>` is `decltype(*declval<T&>())` — the type produced by dereferencing an lvalue of the nullable type. For every nullable above with an `int` payload it is `int&`; for a `const optional<int>` it is `const int&`; for `const int*` it is `const int&`. The return type is then the common type (for `value_or` and `or_invoke`) or the common reference (for `reference_or`) of that dereference type and the alternative.

For `value_or`, `common_type` decays, so the result is a prvalue, and mixed-type uses promote instead of truncating:

nullable `T`	alternative `U`	result `R`
`optional<int>`	`int`	`int`
`expected<int,E>`, `int*`, `shared_ptr<int>`, `unique_ptr<int>`	`int`	`int`
`optional<int>`	`long`	`long`
`optional<int>`	`double`	`double`
`optional<uint16_t>`	`int` (e.g. `-1`)	`int`
`optional<string>`	`string`	`string`

This is the type the equivalent conditional expression would have produced, which is exactly the point: `value_or(m, u)` is the library spelling of `m ? *m : u`, not of `m ? *m : T(u)`.

3.3 Value categories, and observing rather than consuming

As specified, dereference of the nullable is always performed on an lvalue: `iter_reference_t` is computed from `T&`, and the engaged path returns `static_cast<R>(*m)`. A consequence worth stating plainly is that an rvalue nullable does *not* move its payload out; `value_or(std::move(opt), u)` copies the contained value where `std::move(opt).value_or(u)` would move it. The functions observe; they do not consume.

This is a deliberate simplification for R0, and the author seeks LEWG guidance. The alternative — computing the dereference type as `decltype(*std::forward<T>(m))` so that rvalue optionals move — recovers the member function's efficiency for the rvalue case, at the cost of making the return type depend on the value category of the nullable and of introducing a moved-from path that is harder to reason about under the dangling analysis of `reference_or`. The simpler rule has a simple statement: the result is what the lvalue conditional expression would have produced. Performance-sensitive consuming uses retain the member functions.

3.4 `reference_or`

`reference_or` computes `common_reference_t<iter_reference_t<T>, U&&>` and returns it, binding the result to the contained value or to the alternative without copying either. Const qualification flows through the computation as it should: a `const optional<int>` or a `const int&` alternative each force `const int&` as the result; an `int*` with an `int&` alternative yields `int&`, writable through, and mutation through the result reaches the original object — the function introduces no copies.

The danger of returning references is binding one to a temporary, and that is now statically detectable. The implementation carries two guards:

```
static_assert(!std::reference_constructs_from_temporary_v<R, U>);
static_assert(!std::reference_constructs_from_temporary_v<R, T&>);
```

The first rejects the alternative materializing a temporary: in `reference_or(opt, 0)` the computed common reference of `int&` and `int&&` is `const int&`, which would bind to a temporary materialized from the literal, and the call is ill-formed. The second rejects the symmetric case where the conversion of `*m` to `R` would materialize a temporary. Together they admit exactly the calls whose result refers to an object that outlives the expression, and reject at compile time the calls that core language lifetime rules would quietly accept and then dangle.

Whether these should be `static_asserts` or constraints is a design question for LEWG. As `static_asserts`, a dangling call is a hard, clearly-diagnosed error wherever it occurs — including inside an overload-resolution context, where it is an error rather than a removed candidate. As constraints, the overload disappears SFINAE-style, which composes better with generic code probing for usability but turns a lifetime bug into a silent "no matching function" or, worse, a fallback to a different overload. The author's position is that a dangling `reference_or` call is a bug to be reported, not a candidate to be quietly discarded, and R0 proposes the `static_assert` behavior (specified as *Mandates*).

The name is also open. The Tokyo discussion of the P2988 homework concluded that `value_or` is not an available name for a reference-returning function, and suggested `ref_or`; this paper spells it `reference_or`, matching the library's general preference for unabbreviated names, and will bring a naming table for a poll.

3.5 `or_invoke` and laziness

`value_or` and `reference_or` evaluate their alternative unconditionally — it is an ordinary function argument. When the alternative is expensive to construct, that cost is unwelcome on the engaged path, and the established cure is to pass a nullary invocable that is called only when needed. `or_invoke(m, f)` returns `*m` converted

to R when m is engaged and f() converted to R otherwise, with R = common_type_t<iter_reference_t<T>, invoke_result_t<I>>; the invocable is not called on the engaged path.

The obvious name, or_else, is taken: std::optional::or_else returns an optional, re-wrapping the result, and giving the same name to a value-producing free function would make x.or_else(f) and or_else(x, f) mean different things. [P3413R0] faces the same collision for its member function and chooses value_or_else. For the free function this paper proposes or_invoke, and will poll the name alongside value_or_else and or_eval.

3.6 Why free functions

The member-function route is closed twice over. For std::optional and std::expected the existing value_or cannot change its return type, and a second member with corrected semantics would have to coexist with the first forever, on each class, kept in sync by hand. For pointers — raw and smart — there are no members to add at all. A constrained free function template is the only form that can state the semantics once and apply them to every type speaking the protocol, including user-defined nullables, which model the concept simply by being contextually convertible to bool and dereferenceable.

These are ordinary function templates, not customization point objects. There is nothing to customize: the semantics are fully determined by the protocol, and a type that wants different behavior for "value or default" is not modeling nullable, it is doing something else. The usual argument for a CPO — dispatching to a type's own implementation — is therefore absent. Whether the names should nonetheless inhibit ADL is left to LEWG.

3.7 std::expected, and the error

std::expected<T,E> models nullable: both bool(e) and *e are valid on a const expected. Consequently value_or(e, u) works, and discards the error, exactly as the member value_or does. This is by design: the moment the question is "the value, or this instead," the error has been decided to be uninteresting. Uses that need the error have error_or and the monadic interface on the member side; no error-observing overload is proposed here.

3.8 What about yield_if

[P2988R12] forecast a fourth function, yield_if, alongside the three proposed here. yield_if produces a view, not a value; it belongs to the ranges layer and remains with the [P1255R13] work. It is explicitly not part of this proposal.

4 Relation to P3413R0

[P3413R0] ("A more flexible optional::value_or (else!)"), reviving [P2218R0], proposes member functions value_or_construct — constructing the fallback T lazily and in place from forwarded arguments, with initializer_list overloads — and value_or_else — taking a lazily evaluated nullary callable — on both std::optional and std::expected. The two papers aim at the same member function and fix different defects, and LEWG should see them together.

P3413 fixes the *cost* of the alternative: with value_or_construct the fallback object is not constructed at all on the engaged path, and value_or_else defers arbitrary computation. It deliberately does not touch the return type — everything still returns T, with the alternative still required to convert to T — so the truncation pitfall, the forced copies, and the impossibility of reference results all remain. It is also necessarily member-by-member: optional and expected each grow six functions, and pointers of any kind get nothing. On optional<T&> it explicitly defers to P2988.

This paper fixes the *type* of the answer and the *reach* of the idiom: results are computed via common_type/common_reference rather than funneled through T, reference results become possible and dangling-checked, and one set of free functions covers every nullable type. The lazy case overlaps — or_invoke and member value_or_else differ in form (free vs. member), reach (any nullable vs. optional/expected), and return type (common_type vs. T). P3413's value_or_construct has no direct analogue here; its in-place construction is expressible as or_invoke(m, [&]{ return T(args...); }), though without the initializer_list convenience, and a free or_construct could be added later if wanted.

	P3413R0	this paper
form	members	free functions
applies to	`optional`, `expected`	any `nullable` (incl. pointers)
return type	`T`	`common_type` / `common_reference`
truncation pitfall	unchanged	fixed
reference results	no	`reference_or`, dangling-checked
lazy alternative	`value_or_else`	`or_invoke`
in-place fallback construction	`value_or_construct`	via lambda; no direct analogue
rvalue nullable moves payload	yes (`&&` overloads)	no (observes only; see Design)
`optional<T&>`	deferred to P2988	covered by the concept

The proposals are not mutually exclusive: adopting P3413 makes the member interface more complete, and adopting this paper makes the corrected semantics available generically. But where they overlap — the lazy alternative — the committee should choose deliberately whether the long-term spelling is a per-class member returning `T` or a generic free function returning the common type, rather than adopting both names by accident.

5 Principles for Reification of Design

The same principles that governed [P2988R12] govern here.

Errors that can be detected at compile time must be detected at compile time. The concept rejects non-nullable arguments at the constraint; `reference_or` rejects every call whose result would bind to a temporary via `reference_constructs_from_temporary_v`, on both the alternative path and the contained-value path.

Never knowingly produce a dangling reference. The functions take no address and extend no lifetime; `reference_or`'s result always refers to an object that existed before the call. The rejected cases are precisely those where the core language would have materialized a temporary: a prvalue alternative whose common reference with the dereference type is a const reference (`reference_or(opt, 0)`), and conversions of the contained value to the result type that materialize.

Value-producing functions never dangle by construction. `value_or` and `or_invoke` return prvalues of a decayed common type; there is no reference in their result to check.

These principles are enforced by a negative-compilation test suite in the reference implementation: each forbidden combination is a translation unit that must fail to compile, with the diagnostic matched against the constraint or `static_assert` that is required to fire.

6 Implementation Experience

The reference implementation is `beman.free_value_or` [Downey_free_value_or]. The implementation floor is C++23, required for `reference_constructs_from_temporary_v`; the functions are otherwise C++20 material.

The test suite exercises the matrix of: nullable types `std::optional<T>`, `std::expected<T,E>`, `T*`, `std::shared_ptr<T>`, `std::unique_ptr<T>`, and (gated on a C++26 toolchain, via the Beman `optional` reference implementation of P2988 [The_Beman_Project_beman_optional]) `std::optional<T&>`; engaged and disengaged; the nullable as lvalue, const lvalue, and rvalue; the alternative as lvalue and rvalue; mixed-type combinations; constant evaluation of all three functions; laziness of `or_invoke` and eagerness of `value_or`; and negative-compilation cases for both concept violations and dangling rejection. Reference identity and mutation round-trips confirm that `reference_or` introduces no copies: writes through its result reach the contained object when engaged and the alternative when not.

Verified `common_reference` results for `reference_or`, showing const propagation:

nullable (lvalue)	alternative (lvalue)	result
`optional<int>&`	`int&`	`int&`
`const optional<int>&`	`int&`	`const int&`
`optional<int>&`	`const int&`	`const int&`
`expected<int,int>&`	`int&`	`int&`
`int*`	`int&`	`int&`
`shared_ptr<int>&`	`int&`	`int&`
`optional<string>&`	`string&`	`string&`

The current implementation, in full:

```
template <class T>
concept nullable = requires(const T t) {
    bool(t);
    *(t);
};

template <nullable T, class U,
          class R = std::common_reference_t<std::iter_reference_t<T>, U&&>>
constexpr auto reference_or(T&& m, U&& u) -> R {
    static_assert(!std::reference_constructs_from_temporary_v<R, U>);
    static_assert(!std::reference_constructs_from_temporary_v<R, T&>);
    return bool(m) ? static_cast<R>(*m) : static_cast<R>((U&&)u);
}

template <nullable T, class U,
          class R = std::common_type_t<std::iter_reference_t<T>, U&&>>
constexpr auto value_or(T&& m, U&& u) -> R {
    return bool(m) ? static_cast<R>(*m) : static_cast<R>(std::forward<U>(u));
}

template <nullable T, class I,
          class R = std::common_type_t<std::iter_reference_t<T>,
                                       std::invoke_result_t<I>>>
constexpr auto or_invoke(T&& m, I&& invocable) -> R {
    return bool(m) ? static_cast<R>(*m) : static_cast<R>(invocable());
}
```

7 Proposal

Add to the standard library three constrained free function templates — `value_or`, `reference_or`, and `or_invoke` — and the exposition-level concept that constrains them, with the semantics described above, targeting C++29. The functions should be usable on any type that is contextually convertible to `bool` and dereferenceable, including `std::optional`, `std::expected`, raw pointers, `std::shared_ptr`, `std::unique_ptr`, and `std::optional<T&>`.

Open questions on which LEWG guidance is sought before wording review:

- whether the dangling guards of `reference_or` are *Mandates* (the proposed default) or *Constraints*;
- the names `reference_or` (vs. `ref_or`) and `or_invoke` (vs. `value_or_else`, `or_eval`);
- whether the concept requires `is_object_v` of the referent, excluding function pointers at the constraint;
- whether rvalue nullables should move their payload (see Design);
- whether the concept is exposition-only or named and public;
- header placement: `<optional>` is where users will look, `<utility>` matches the genericity, a new header is cleanest for freestanding; the functions should be freestanding in any case.

8 Wording

Initial draft wording, relative to [N5008], to be completed after LEWG design review. Placement (shown here as `<utility>`) is provisional.

Add to the header synopsis:

```
namespace std {
  template<class T>
    concept nullable = requires(const T t) {   // exposition only
      bool(t);
      *(t);
    };

  template<nullable T, class U>
    constexpr common_reference_t<iter_reference_t<T>, U&&>
      reference_or(T&& m, U&& u);

  template<nullable T, class U>
    constexpr common_type_t<iter_reference_t<T>, U&&>
      value_or(T&& m, U&& u);

  template<nullable T, class I>
    constexpr common_type_t<iter_reference_t<T>, invoke_result_t<I>>
      or_invoke(T&& m, I&& invocable);
}
```

```
template<nullable T, class U>
  constexpr common_reference_t<iter_reference_t<T>, U&&>
    reference_or(T&& m, U&& u);
```

1 Let `R` be `common_reference_t<iter_reference_t<T>, U&&>`.

2 *Mandates*: `reference_constructs_from_temporary_v<R, U>` is `false` and `reference_constructs_from_temporary_v<R, T&>` is `false`.

3 *Effects*: Equivalent to: `return bool(m) ? static_cast<R>(*m) : static_cast<R>(std::forward<U>(u));`

```
template<nullable T, class U>
  constexpr common_type_t<iter_reference_t<T>, U&&>
    value_or(T&& m, U&& u);
```

4 Let `R` be `common_type_t<iter_reference_t<T>, U&&>`.

5 *Effects*: Equivalent to: `return bool(m) ? static_cast<R>(*m) : static_cast<R>(std::forward<U>(u));`

```
template<nullable T, class I>
  constexpr common_type_t<iter_reference_t<T>, invoke_result_t<I>>
    or_invoke(T&& m, I&& invocable);
```

6 Let `R` be `common_type_t<iter_reference_t<T>, invoke_result_t<I>>`.

7 *Effects*: Equivalent to: `return bool(m) ? static_cast<R>(*m) : static_cast<R>(invocable());`

8.1 Feature test macro

Add a new feature test macro `__cpp_lib_value_or`, set to the date of adoption.

9 Impact on the standard

A pure library extension. No existing class, function, or wording changes; no ABI impact. The facilities require C++23 library support (`reference_constructs_from_temporary_v`).

10 Acknowledgments

Thanks to Peter Sommerlad, co-author of [P2988R12], where the member `value_or` question was fought to a standstill, and to the LEWG reviewers of P1255 and P2988 whose objections shaped these functions. The Tokyo review homework on `value_or` alternatives surveyed the implementation divergence reported here. Thanks to Marc Mutz and Corentin Jabot, whose [P2218R0] and [P3413R0] map the member-function side of this design space.

11 Document history

— **R0** 2026-06-12 — Initial draft. The free-function follow-on promised in [P2988R12], continuing [P1255R13].

References

[Downey_free_value_or] Stephen Downey. beman.free_value_or. https://github.com/steve-downey/free_value_or.

[N5008] Thomas Köppe. N5008: Working draft, programming languages — c++. https://wg21.link/n5008, 3 2025.

[P1255R13] Steve Downey. P1255R13: A view of 0 or 1 elements: views::nullable and a concept to constrain maybes. https://wg21.link/p1255r13, 5 2024.

[P2218R0] Marc Mutz. P2218R0: More flexible optional::value_or(). https://wg21.link/p2218r0, 9 2020.

[P2255R2] Tim Song. P2255R2: A type trait to detect reference binding to temporary. https://wg21.link/p2255r2, 10 2021.

[P2988R12] Steve Downey and Peter Sommerlad. P2988R12: std::optional<t&>. https://wg21.link/p2988r12, 4 2025.

[P3413R0] Corentin Jabot. P3413R0: A more flexible optional::value_or (else!). https://wg21.link/p3413r0, 10 2024.

[The_Beman_Project_beman_optional] The Beman Project. beman.optional. https://github.com/bemanproject/optional.